

Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Cloud-Native Automated Deployment Data 2026/5/21
Class 2023 Applied Computing Name Wang Shiyu ID 202383910020

一、 Experimental purpose

The primary goal of this experiment is to introduce students to the core concepts of Cloud Computing through hands-on practice with Platform-as-a-Service (PaaS) and Automation.

二、 Experiment Contents

This lab consists of five integrated modules designed to simulate a professional cloud deployment lifecycle.

三、 Detailed steps of the experiment

Lab Report Requirements: Clearly present the procedure step by step, with corresponding screenshots provided for each stage. **Screenshots must demonstrate that they were taken on your own computer**, for example by including visible system time, desktop environment, or other identifiable personal elements.

Step 1: Resource Acquisition

Log in to GitHub.

Navigate to the repository: <https://github.com/yhldrf/cloud-computing-lab-github> and click Fork.

Cloud Concept: You are creating a private instance of the project in the cloud ecosystem.

Step 2: Enabling the Hosting Service

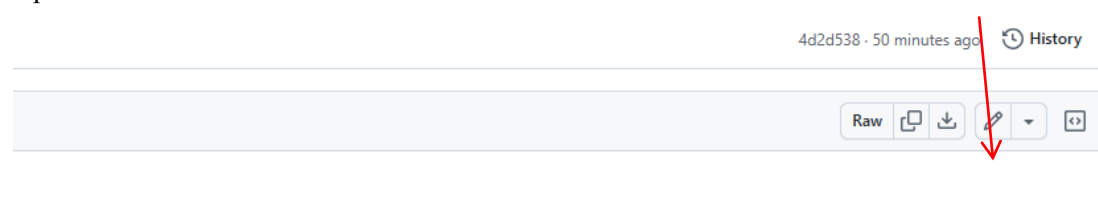
Go to Settings > Pages.

Under Build and deployment > Source, select "GitHub Actions". After that, leave this page.

Cloud Concept: You are switching from "Manual Upload" to "Managed Pipeline Service."

Step 3: Triggering Cloud Automation

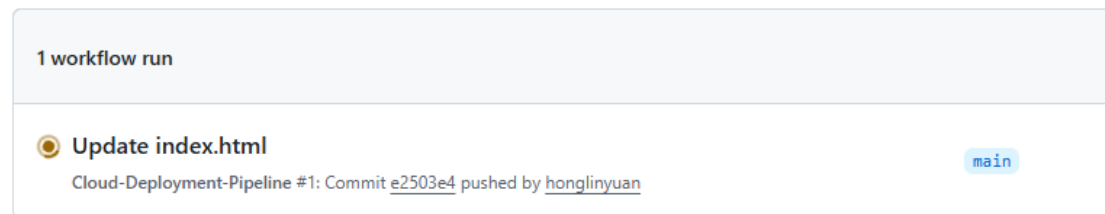
Open index.html in the browser editor.



Change [Enter Your Name Here] to your actual name.
Click Commit Changes.
Immediately click the Actions tab at the top of the page.
Observation: Watch the "Cloud-Deployment-Pipeline" start.

All workflows

Showing runs from all workflows



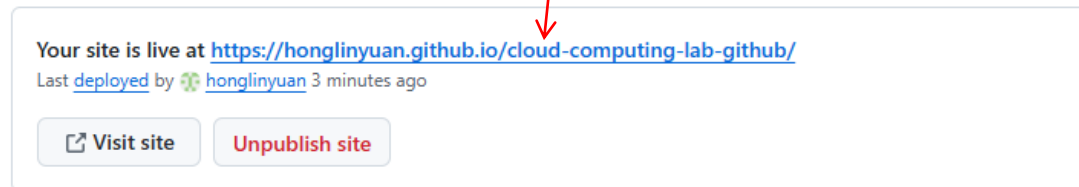
Cloud Concept: GitHub is spinning up a Virtual Machine (Ubuntu) to process your request. This is "On-demand Computing."

Step 4: Verification & Global Access

Once the Action finishes (green check), go back to Settings > Pages to find your Live URL.

GitHub Pages

GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.



Open the URL on your laptop.

Cloud Concept: The site is deployed via a PaaS (Platform as a Service) model. The cloud provider manages the underlying infrastructure and global delivery, allowing us to focus only on our content.

Step5: Content Personalization & Elements (optional)

Task: Beyond changing your name, modify the following HTML elements to see how the cloud updates different types of content.

✓ Change the Theme Color (CSS Styles)

Find the<style>block at the top and change the background color or font size.

Action:

<style>

```
body {  
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
    text-align: center;
```

```
padding-top: 50px;
background-color: #f4f4f9; /* Add a light grey background */
}
.status { color: #007bff; font-weight: bold; } /* Change status to Blue */
</style>
```

Cloud Concept: You are modifying the User Interface (UI) settings stored in the cloud.

✓ Multimedia Integration (Images)

Demonstrate how the cloud handles external resources.

What to add: Antag inside the<body>.

Example:

```

```

Observation: Notice that as soon as you click "Commit," the Actions tab triggers a new build automatically. This is Continuous Deployment (CD).

四、 Experiment Process

1) Step 1: Resource Acquisition

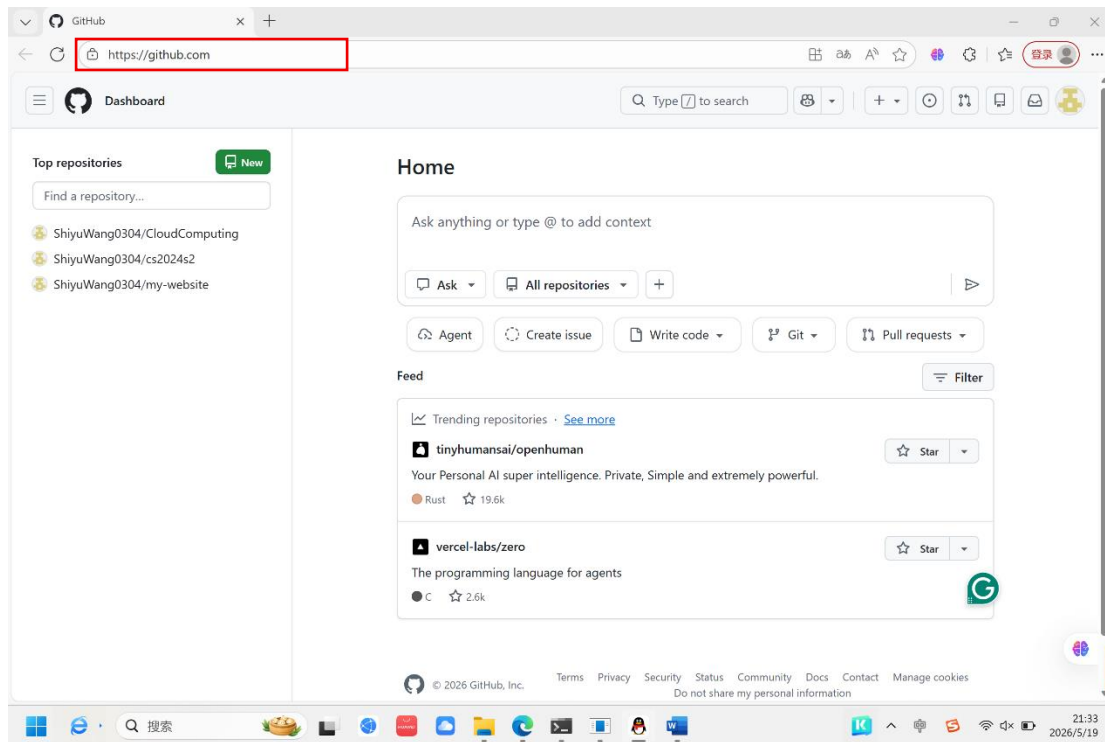
To create a private instance of an existing cloud-based project by forking a GitHub repository, demonstrating the concept of cloud resource provisioning.

This matters in Cloud Computing.

| Traditional Approach | Cloud-Based Fork |
|----------------------------------|---------------------------------|
| Download ZIP, extract, re-upload | One-click operation |
| Manual file management | Automatic replication |
| Local storage required | Cloud-native storage |
| No version history preserved | Complete Git history maintained |
| No connection to original | Link to upstream maintained |

(1) Logged into GitHub Account

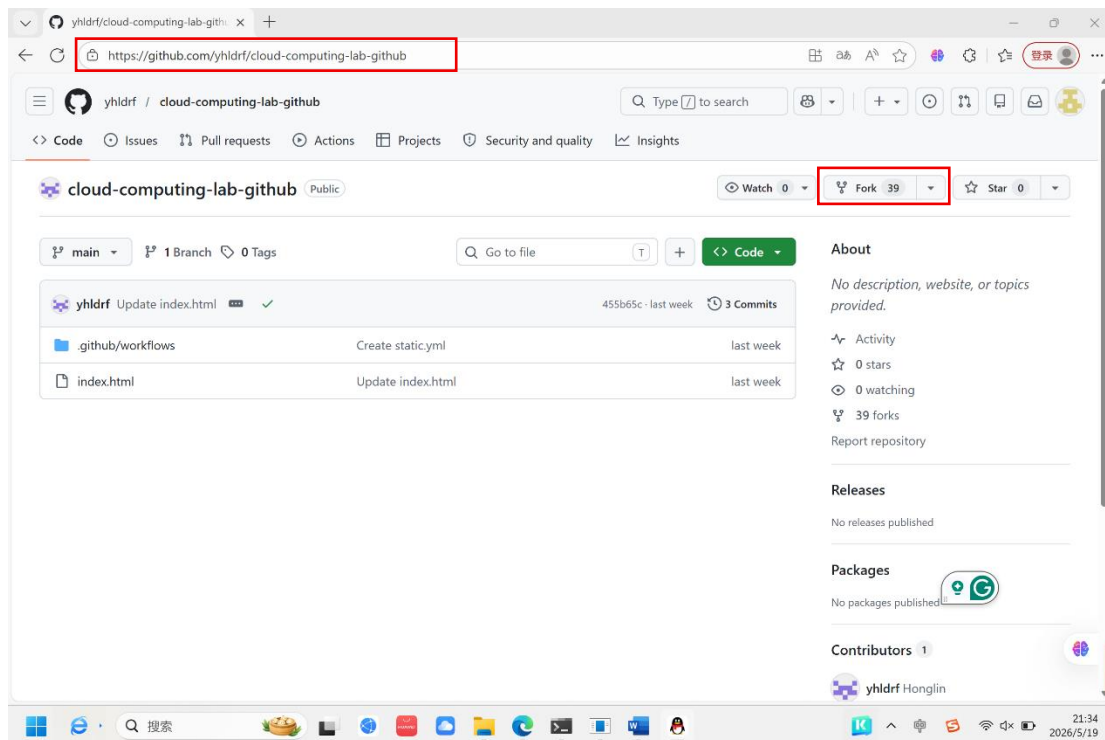
Navigated to <https://github.com>, and entered credentials and successfully authenticated into the GitHub platform.



(2) Executed Fork Operation

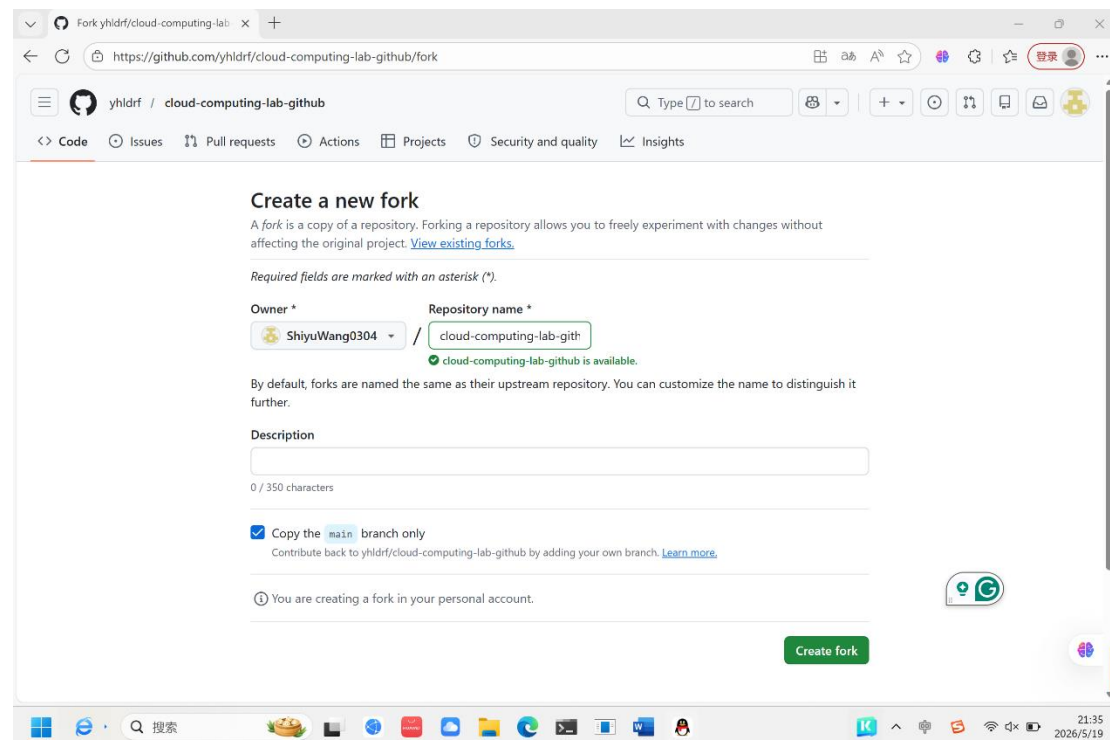
Accessed the target repository URL: <https://github.com/yhldrf/cloud-computing-lab-github>. This repository contains a pre-configured static website template with HTML, CSS, and automated deployment configurations.

Then, located the "Fork" button in the upper-right corner of the repository page, next to the Star and Watch buttons,.

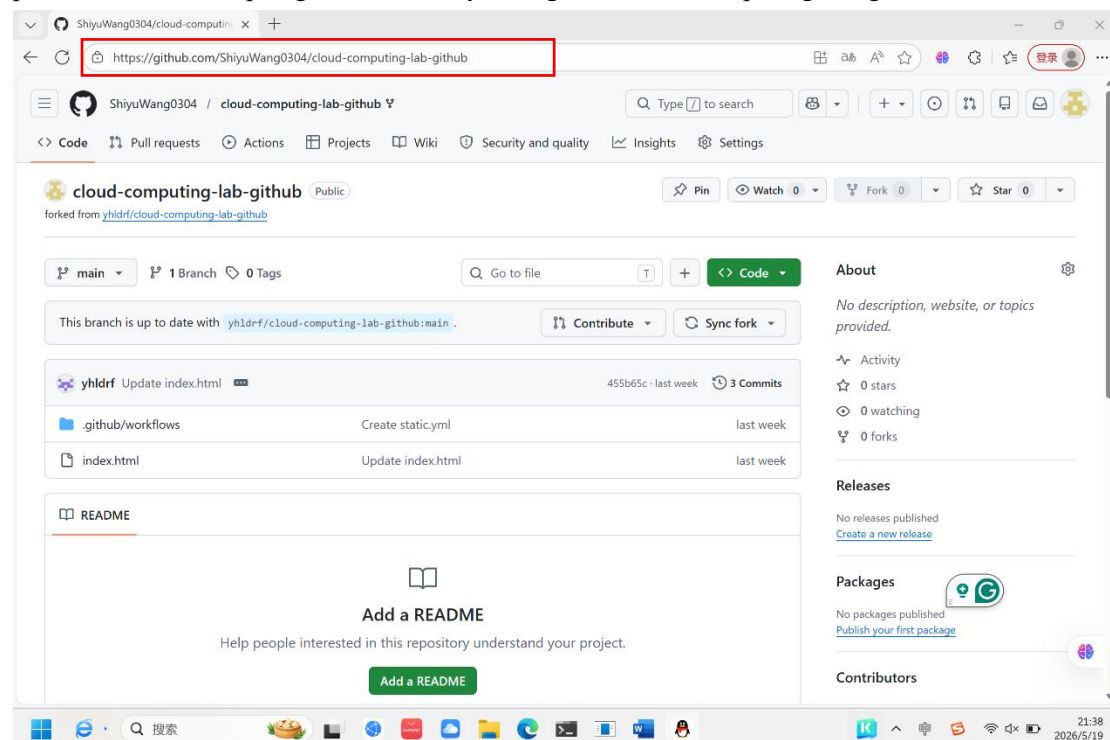


Clicked the "Fork" button to initiate the forking process, and waited approximately 5-10 seconds

for the operation to complete.



Upon completion, the browser automatically redirected to the newly created fork under my personal account: <https://github.com/ShiyuWang0304/cloud-computing-lab-github>.



2) Step 2: Enabling the Hosting Service

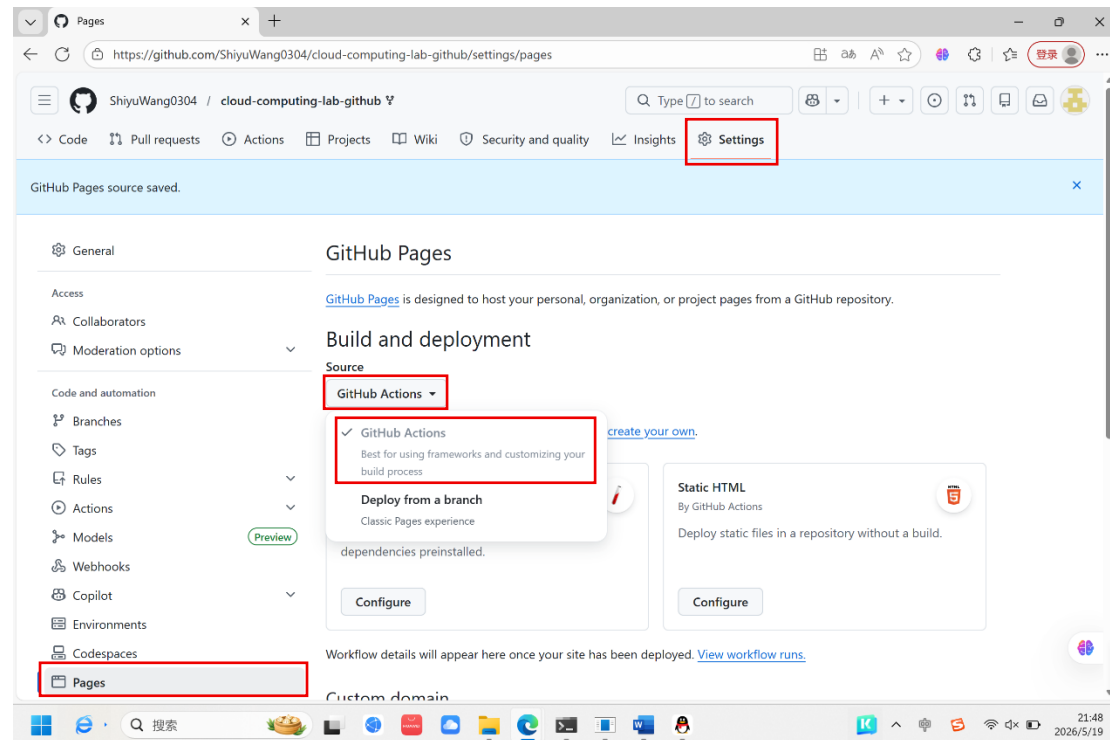
To configure GitHub Pages service with GitHub Actions as the build source, transitioning from manual deployment to an automated managed pipeline service.

(1) Configured Build Source

From the forked repository's main page, clicked on the "Settings" tab located in the top navigation bar. The Settings page provides access to all repository configuration options.

In the left sidebar menu, scrolled down to find the "Pages" option under the "Code and automation" section, and clicked to open the GitHub Pages configuration interface.

Located the "Build and deployment" section, found the "Source" dropdown menu, which had options: "Deploy from a branch" and "GitHub Actions", and changed the selection from "Deploy from a branch" to "GitHub Actions". The page automatically saved the configuration.

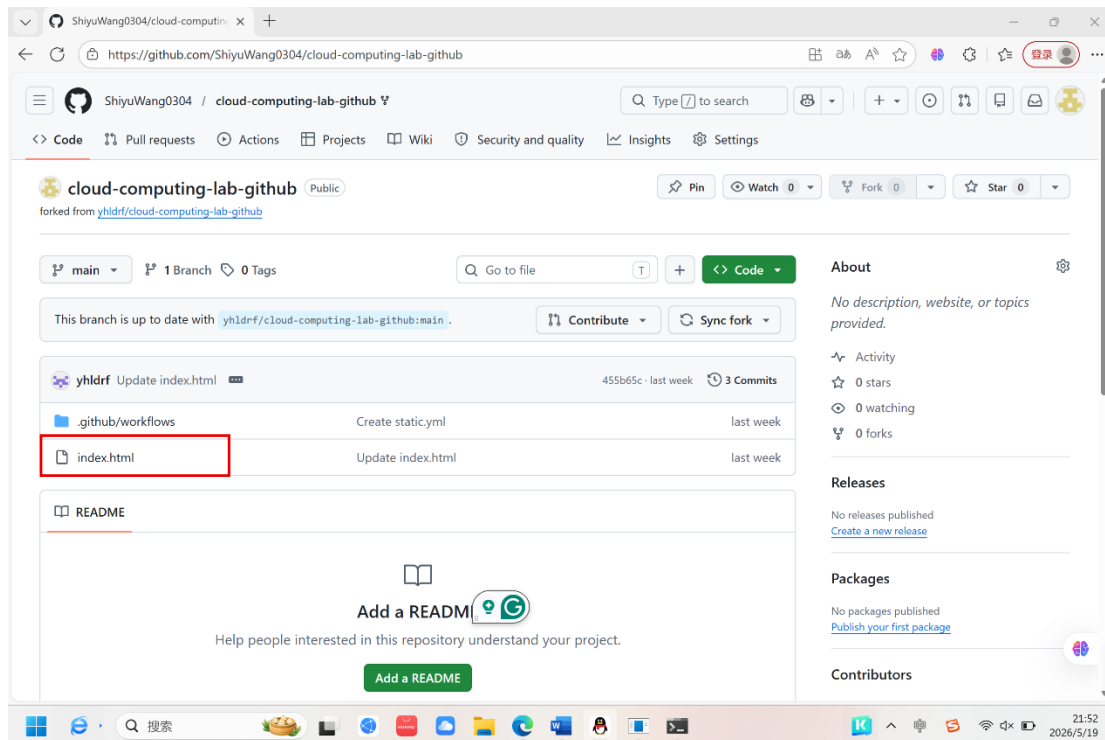


3) Step 3: Triggering Cloud Automation

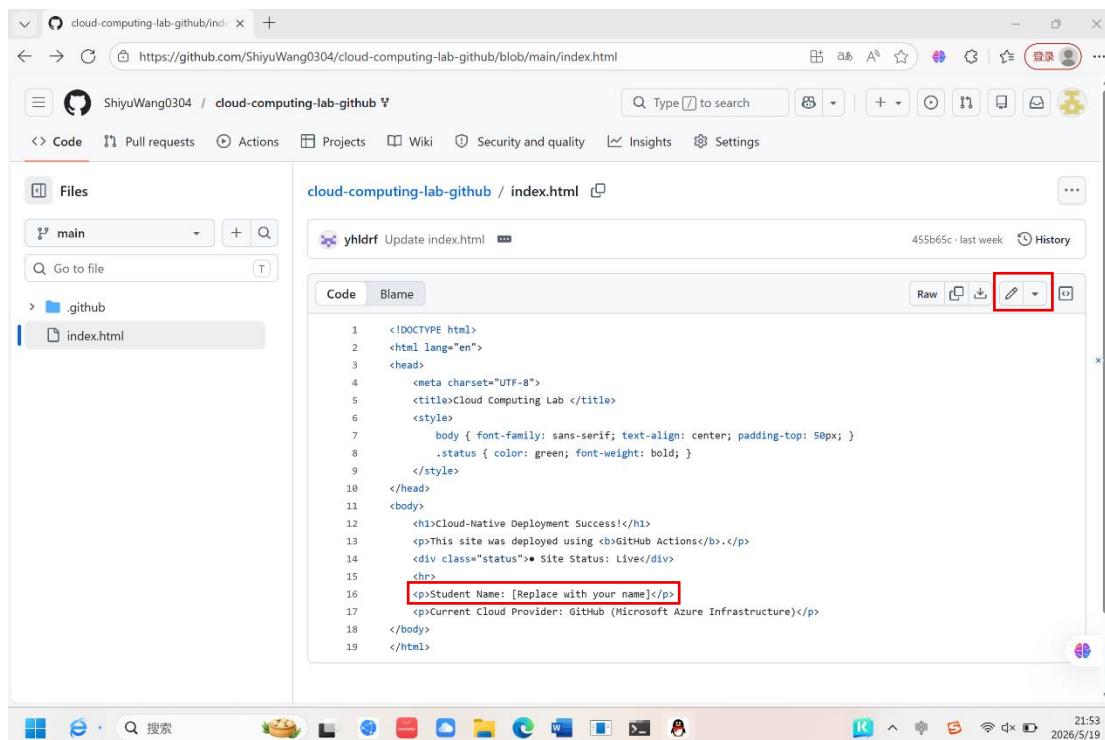
To modify the web application code and observe the automated cloud deployment pipeline in action, experiencing the concept of on-demand computing.

(1) Opened Source File for Editing

From the repository main page, located and clicked on "index.html" file.



Clicked the pencil icon (Edit this file) in the upper-right corner to enter edit mode. The browser-based editor allows direct modification of files without any local development environment.

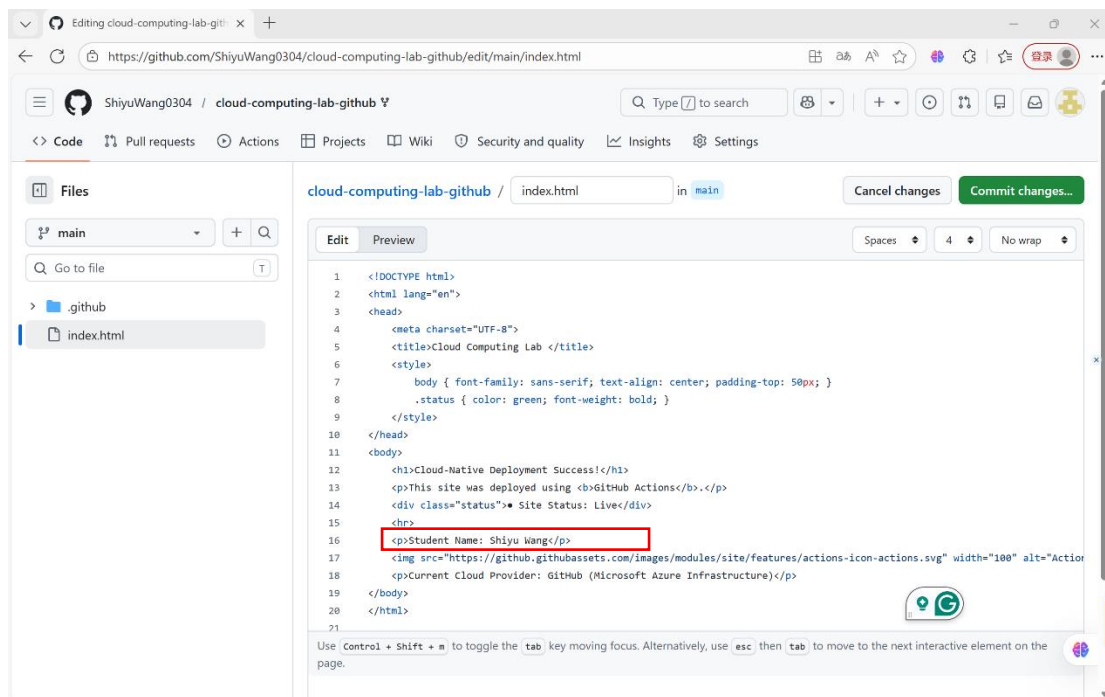


(2) Made Code Modification

Scanned through the HTML code to locate the placeholder text: [Replace with your name]

Replaced the placeholder with actual name: [Shiyu Wang]

This modification personalizes the web page content.

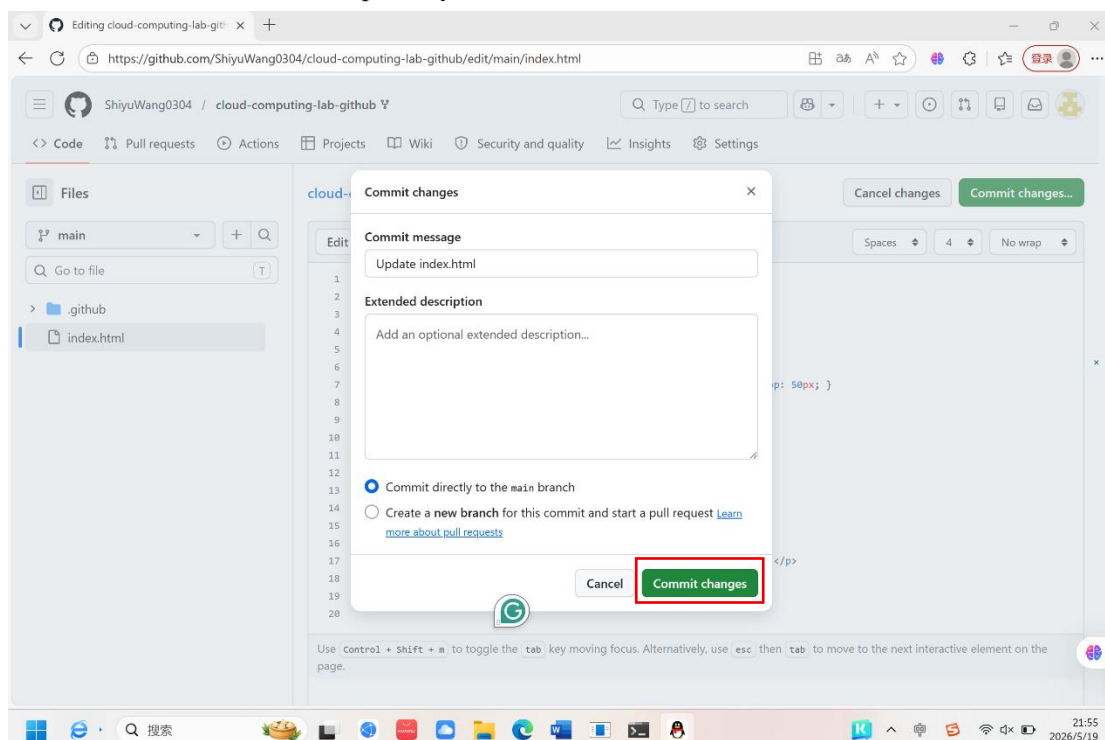


(3) Committed the Changes

Scrolled to the bottom of the edit page, observed the commit interface showing:

- Commit message field (pre-filled with "Update index.html")
- Extended description field (optional)

Accepted the default commit message, clicked the "Commit changes" button (green button), and then the file was saved to the repository's main branch.



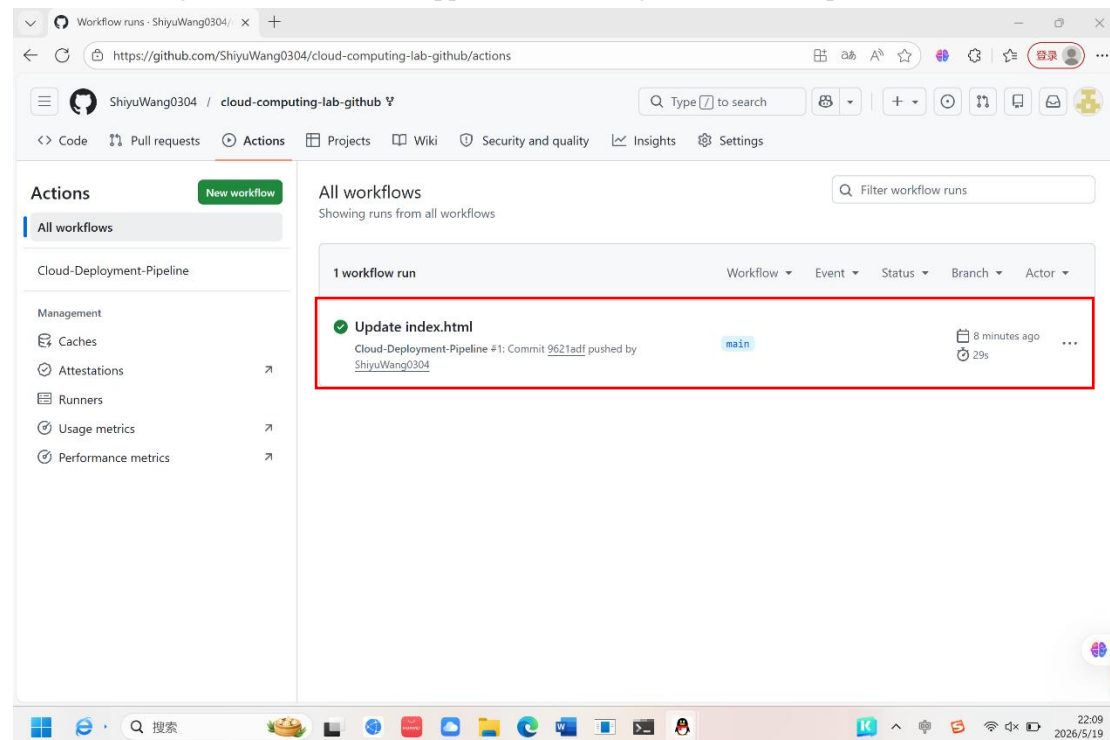
(4) Monitored Automation Pipeline

Immediately after committing, clicked the "Actions" tab in the top navigation bar, observed a new workflow run appearing with the name "Cloud-Deployment-Pipeline", and clicked on the workflow run to view detailed execution logs.

Watched the progress through different stages:

- Setup job
- Checkout repository
- Build and deploy

Waited until a green checkmark (✓) appeared, indicating successful completion



4) Step 4: Verification & Global Access

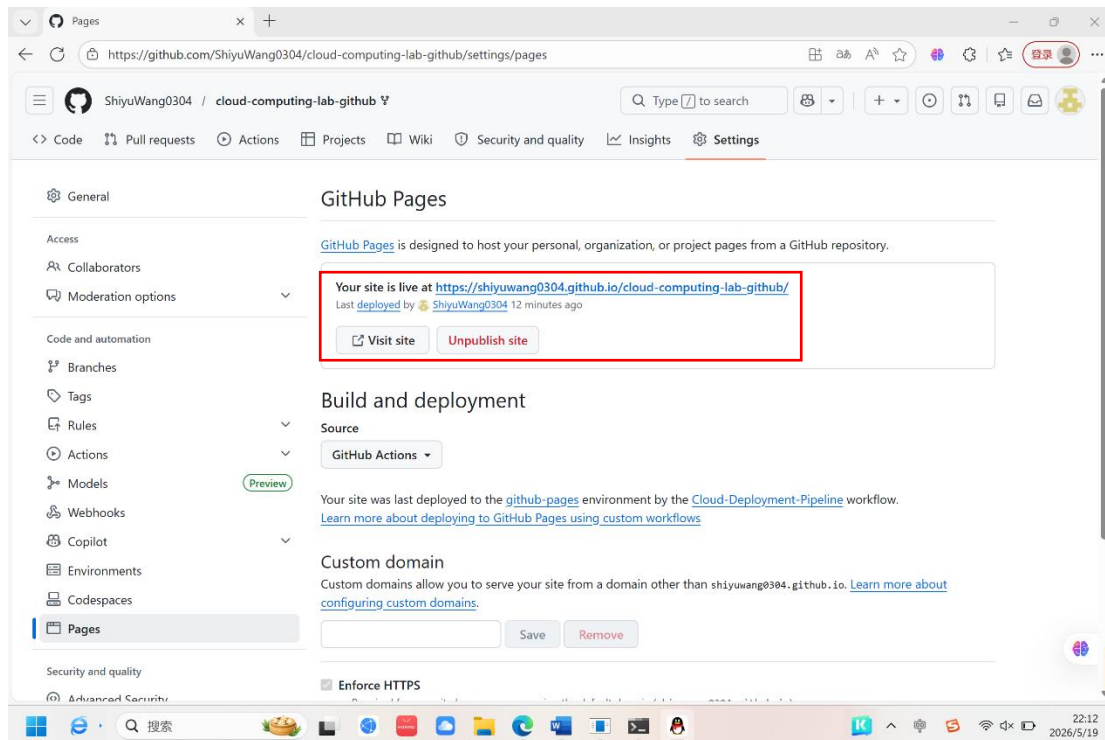
To verify the successful deployment of the website and access it globally through the provided URL, understanding the global content delivery mechanism.

(1) Confirmed Deployment Status

Returned to Settings > Pages after confirming the Actions workflow completed successfully (green checkmark in Actions tab), and then waited for the deployment to finalize.

(2) Located Live URL

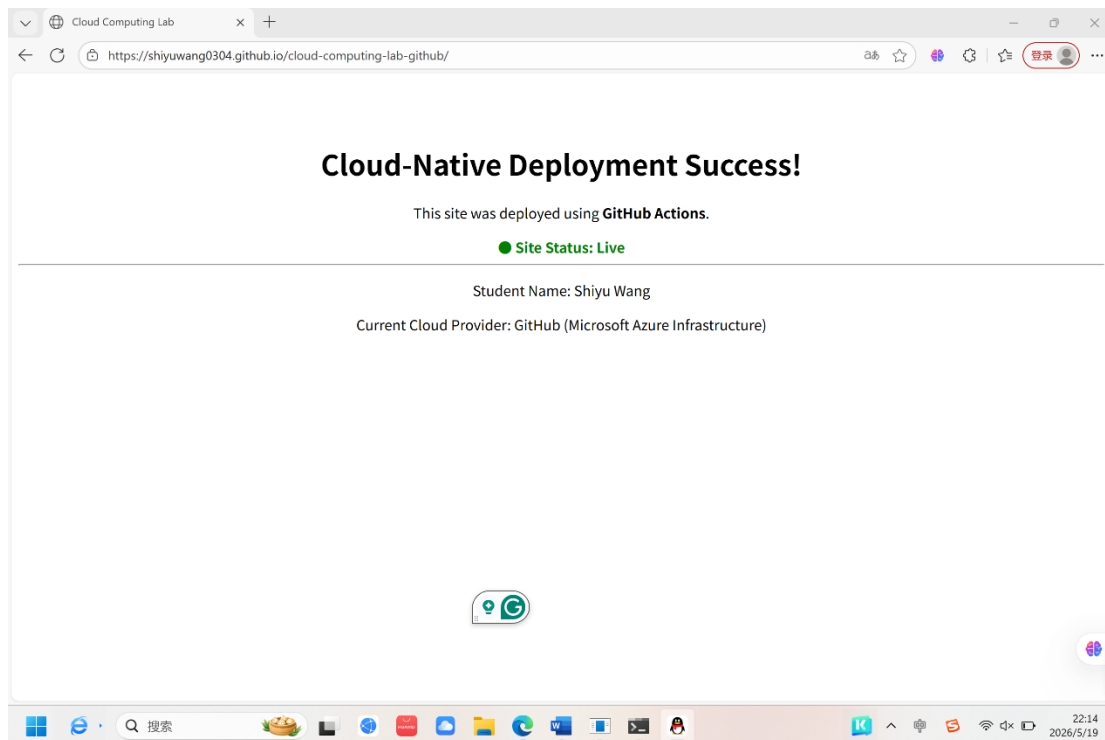
In the Pages settings, found the "Visit site" section at the top of the page. The live URL was displayed in the format: <https://shiyuwang0304.github.io/cloud-computing-lab-github/>. This URL represents the globally accessible endpoint for the deployed website.



(3) Accessed the Deployed Website

Clicked the "Visit site" link, or copied and pasted the URL into a new browser tab. The personalized webpage loaded successfully, displaying:

- The modified name
- The styled content with proper CSS formatting
- All elements rendered correctly



5) Step 5: Content Personalization & Elements

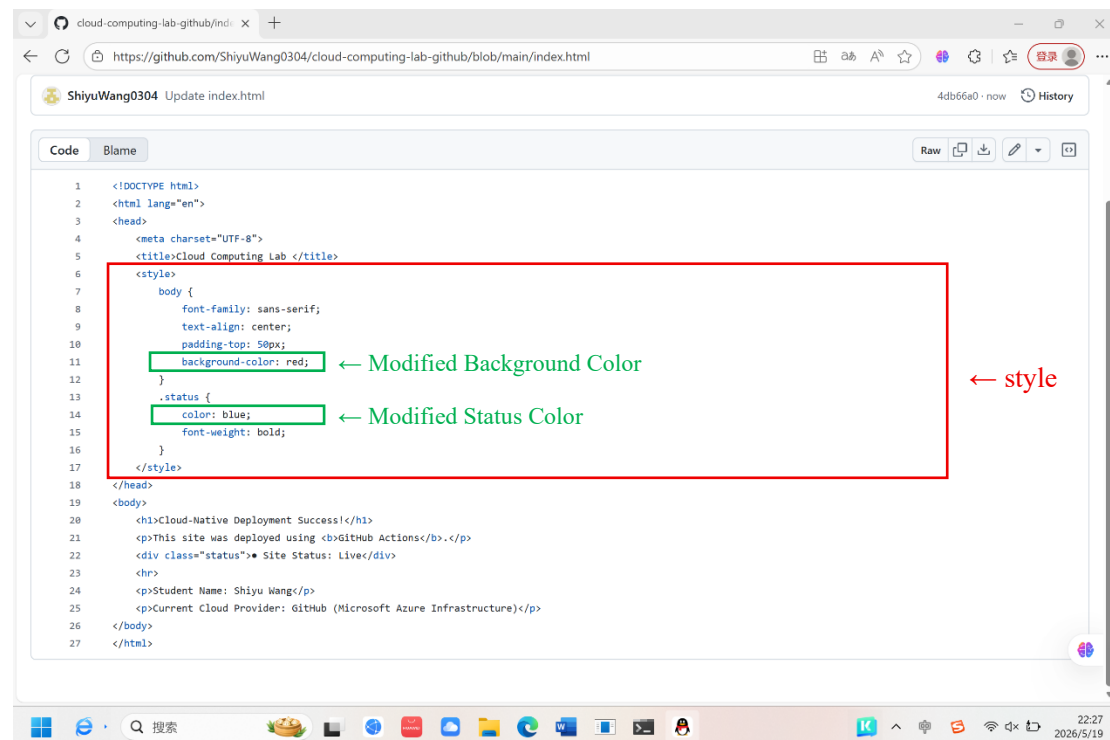
To further customize the web application by modifying CSS styles and adding multimedia elements, demonstrating how cloud platforms handle different types of content updates through continuous deployment.

Part A: Modifying Theme Colors (CSS Styles)

(1) Opened index.html for Editing

Navigated to index.html in the repository, and clicked the pencil icon to enter edit mode.

Located the Style Block to find the <style> tags in the HTML head section:

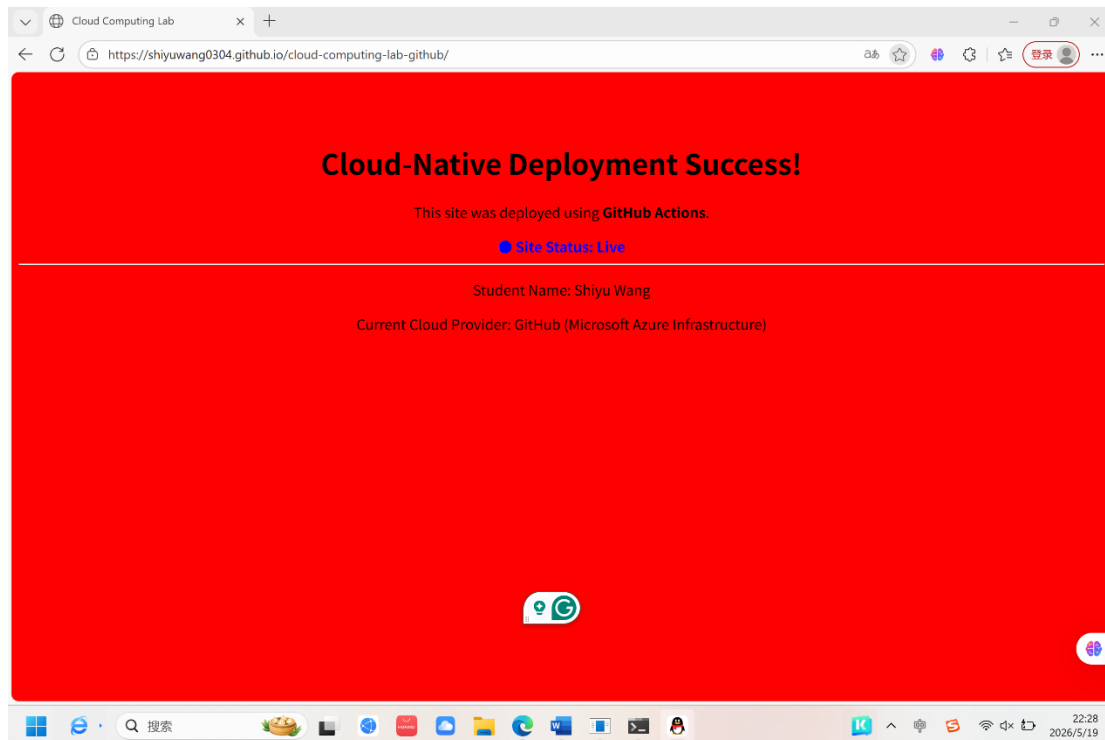


```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Cloud Computing Lab </title>
6   <style>
7     body {
8       font-family: sans-serif;
9       text-align: center;
10      padding-top: 50px;
11      background-color: red;
12    }
13    .status {
14      color: blue;
15      font-weight: bold;
16    }
17  </style>
18 </head>
19 <body>
20   <h1>Cloud-Native Deployment Success!</h1>
21   <p>This site was deployed using <b>GitHub Actions</b>.</p>
22   <div class="status">● Site Status: Live</div>
23   <hr>
24   <p>Student Name: Shiyu Wang</p>
25   <p>Current Cloud Provider: Github (Microsoft Azure Infrastructure)</p>
26 </body>
27 </html>
```

- Modified Background Color: Changed the background color from light grey to a personalized color.
- Modified Status Color: Updated the status text color.

(2) Committed Changes

Added a descriptive commit message: "Update theme colors", committed the changes.

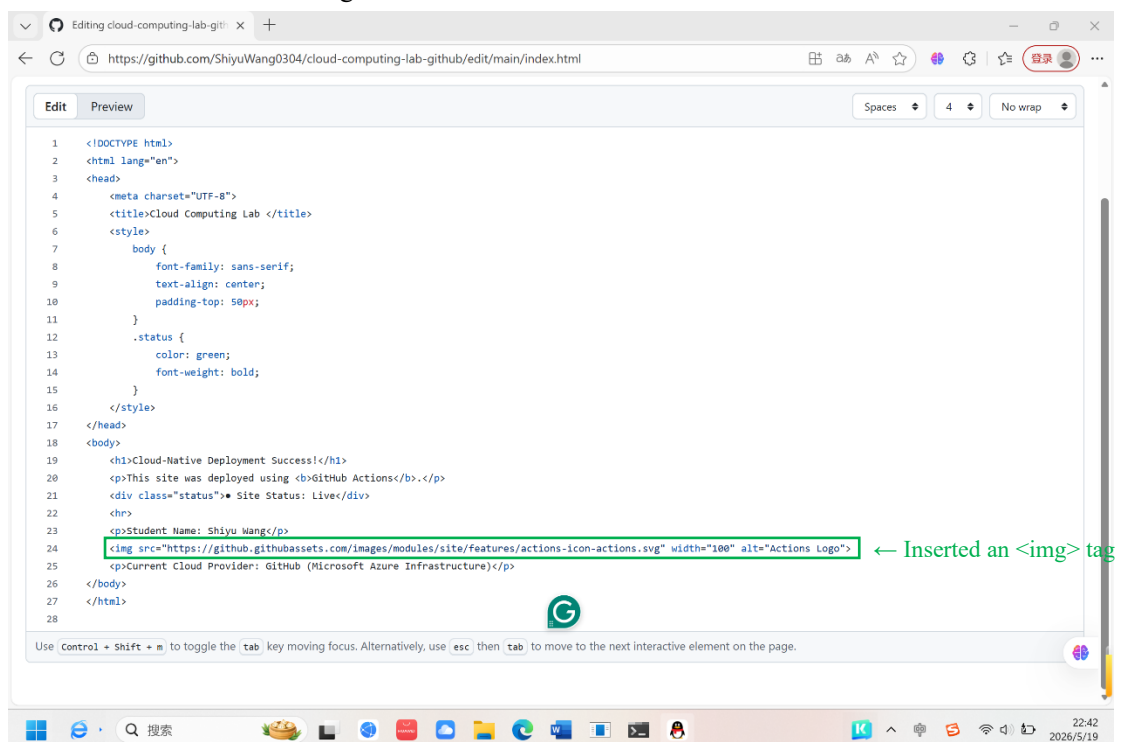


Part B: Adding Multimedia Elements (Images)

(1) Added Image Element

Found the `<body>` section where content is displayed, and identified an appropriate location to insert an image element.

Inserted an `<img>` tag with an external image source: ``

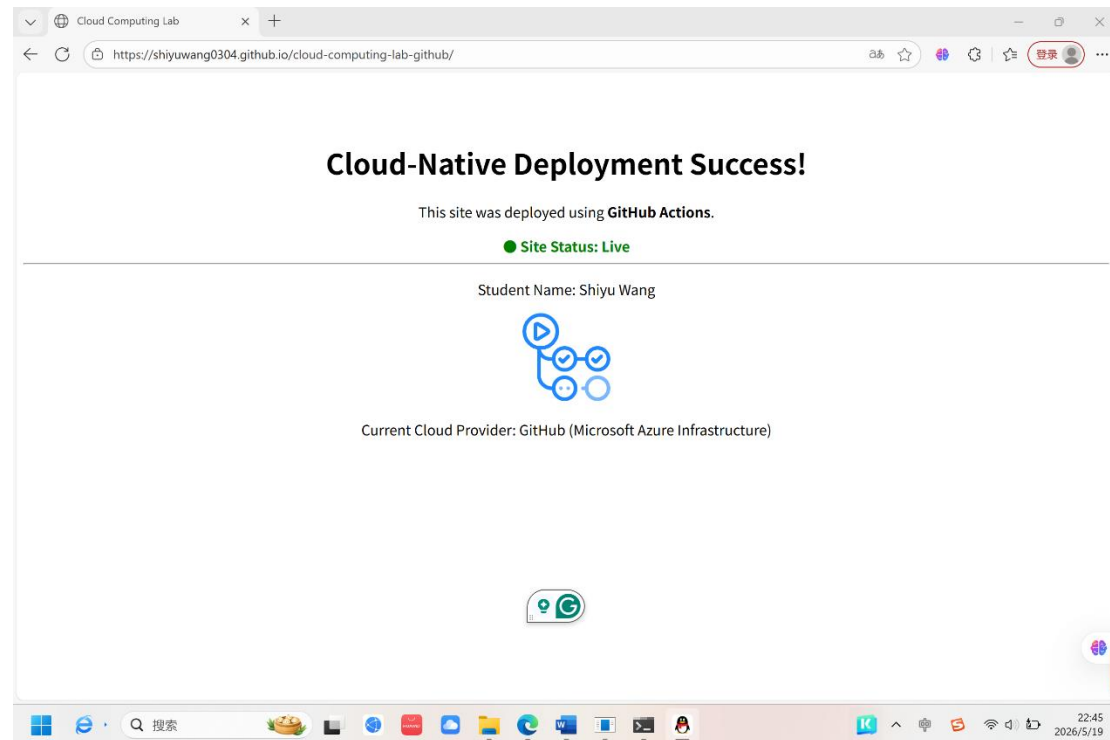


(2) Observed Automatic Rebuild

After committing the image addition, immediately navigated to the Actions tab, and observed that a new workflow was automatically triggered, which demonstrates Continuous Deployment - every change automatically deploys.

After the Actions workflow completed, refreshed the live website. Confirmed that:

- The background color changed
- The status text color updated
- The image appeared correctly



五、 Reflection

1. Where is your website stored?

- 1) My website is stored and hosted on GitHub's cloud infrastructure through GitHub Pages.
- 2) The source code of the website, including index.html, CSS styles, and other project files, is stored in my GitHub repository. After I committed changes to the repository, GitHub Actions automatically built and deployed the website to GitHub Pages.
- 3) The deployed website is not stored on my local computer. Instead, it is hosted on GitHub's servers and can be accessed through a public URL:
<https://shiyuwang0304.github.io/cloud-computing-lab-github/>
- 4) GitHub Pages uses cloud hosting and content delivery mechanisms to make the website accessible from different locations around the world. This reflects the cloud computing

concept of Platform as a Service (PaaS), because GitHub manages the underlying servers, storage, networking, and delivery infrastructure, while I only need to manage my website content.

2. Did you have to install any software?

No, I did not have to install any additional software to complete this experiment.

All operations were completed through a web browser, including:

- Logging in to GitHub
- Forking the repository
- Editing the index.html file online
- Committing changes
- Running GitHub Actions automatically
- Deploying the website through GitHub Pages

I did not need to install Git, a code editor, a web server, or any deployment tool on my local computer. This shows an important advantage of cloud computing and PaaS: the cloud platform provides the development, automation, and hosting environment as an online service.

Because of this, users can focus on modifying and deploying their application content instead of configuring servers or installing software locally.

3. Problems Encountered and Proposed Solutions

| Problem 1: Website did not update immediately after deployment | |
|--|---|
| Problem Description | After committing changes and seeing the GitHub Actions workflow finish successfully, the website might not show the latest content immediately. |
| Possible Reason | This may be caused by browser cache or GitHub Pages/CDN propagation delay. The old version of the webpage may still be temporarily cached. |
| Proposed Solution | <ol style="list-style-type: none">1. Wait for one or two minutes after the workflow finishes.2. Refresh the webpage.3. Use a hard refresh, such as Ctrl + F5.4. Open the website in an incognito/private browser window.5. Confirm that the GitHub Actions workflow has completed successfully with a green check mark. |
| Problem 2: GitHub Actions workflow failed or did not run | |
| Problem Description | Sometimes the workflow may fail, or the Actions page may not show a running workflow after committing changes. |

| | |
|--|---|
| Possible Reason | This may happen if GitHub Pages is not correctly configured to use GitHub Actions, or if the workflow file has errors. |
| Proposed Solution | <ol style="list-style-type: none"> 1. Go to Settings > Pages and confirm that the source is set to GitHub Actions. 2. Check the Actions tab to view the error logs. 3. Read the error message carefully to identify which step failed. 4. Make sure the repository contains the correct workflow configuration files. 5. Commit another small change to trigger the workflow again. |
| Problem 3: HTML or CSS modification caused display problems | |
| Problem Description | When editing index.html, incorrect HTML or CSS syntax may cause the webpage to display incorrectly. |
| Possible Reason | The browser-based editor does not provide as much syntax checking as a professional IDE. Missing tags, wrong quotation marks, or incorrect CSS syntax may affect the final webpage. |
| Proposed Solution | <ol style="list-style-type: none"> 1. Modify the code carefully and avoid deleting important tags. 2. Make small changes each time and test after each commit. 3. Check whether HTML tags such as <code><html></code>, <code><head></code>, <code><style></code>, <code><body></code>, and <code></code> are correctly written. 4. If the page breaks, use GitHub commit history to compare or restore a previous working version. |
| Problem 4: Could not find the live website URL | |
| Problem Description | After the deployment completed, it may be unclear where to find the website URL. |
| Possible Reason | The GitHub Pages URL only appears after Pages is correctly enabled and the deployment is completed. |
| Proposed Solution | <ol style="list-style-type: none"> 1. Go to Settings > Pages. 2. Wait until GitHub Pages finishes publishing the website. 3. Look for the message showing the live site link. 4. Click Visit site to open the deployed website. |